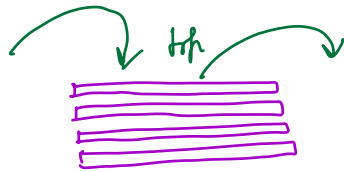
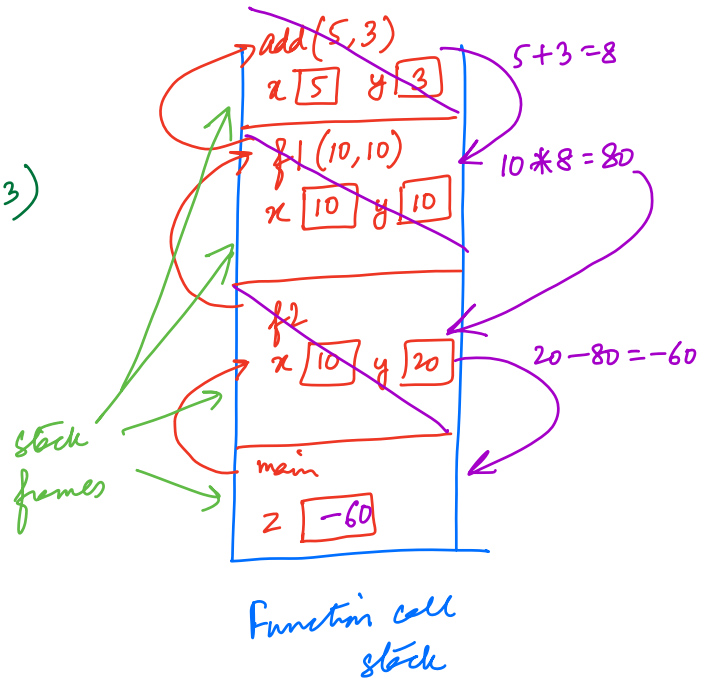


stack $\rightarrow$ of plates



insertion and removal at the top

```
fn add(x, y) {  
    return x + y  
}  
fn f1(x, y) {  
    return x * add(x/2, y/3)  
}  
fn f2(x, y) {  
    return y - f1(x, y/2)  
}  
-- main(..) {  
    ...  
    z = f2(10, 20);  
}
```



```
f5(..) {  
    f6(...)  
    f7(..)  
}
```

```
f2(...) {
```

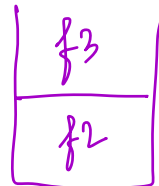
```
    if (x < y)
```

```
        f3(...)
```

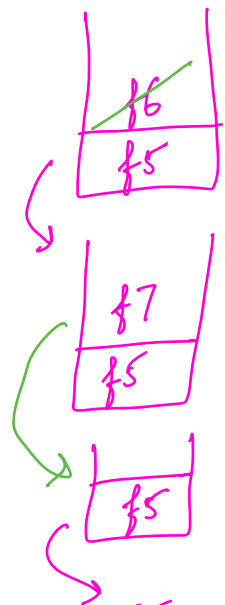
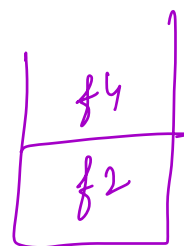
```
    else
```

```
        f4(...)
```

True



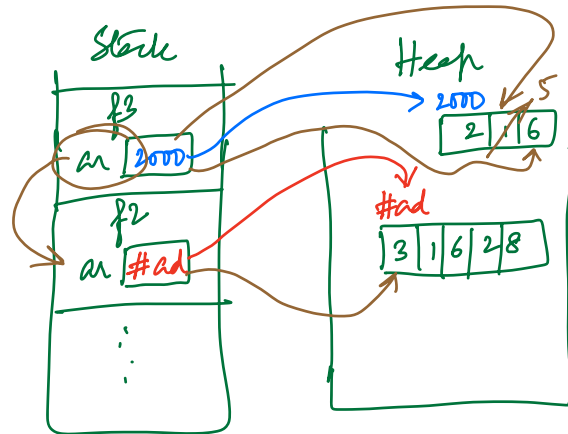
False



2 Types of memory in Java:

- Stack memory: all data of primitive data types, and references
- Heap memory: containers of references.

```
f2(..) {  
    ar → {3, 1, 6, 2, 8}  
    print ar[0] → 3  
}  
f3(..) {  
    ar → {2, 1, 6}  
    print ar[1] → 6  
    ar[1] = 5  
}
```



```
change(a) {
```

```
    a = 50
```

```
}
```

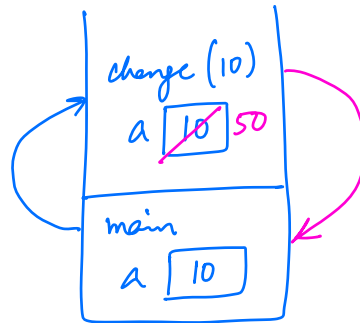
```
main(....) {
```

```
    a = 10
```

```
    change(a)
```

```
    Print(a)
```

```
}
```



```
change(a) {
```

```
    a[0] = 50
```

```
}
```

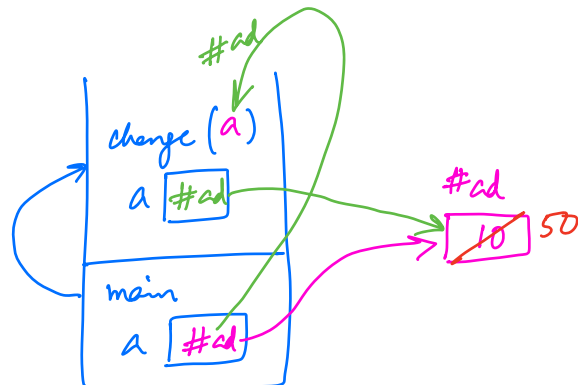
```
main(....) {
```

```
    a[] = {10}
```

```
    change(a)
```

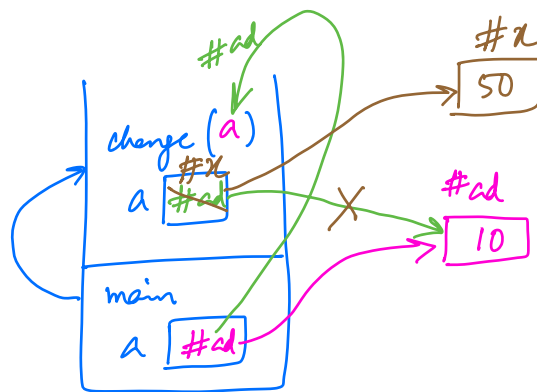
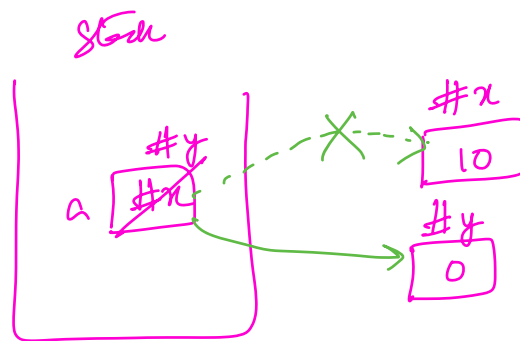
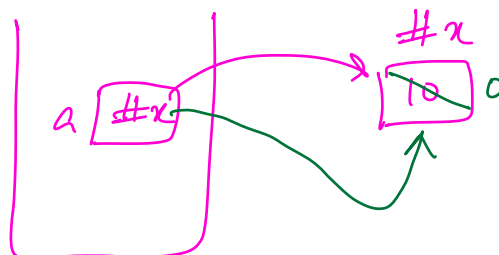
```
    Print(a[0])
```

```
}
```



```
change(a) {
    a = new int[1];
    a[0] = 50
}

main (....) {
    a[] = {10}
    change(a)
    print(a[0])
}
```


$$a = \{10\}$$
$$a = \{0\}$$

$$a = \{10\}$$
$$a[0] = 0$$


```

change(a) {
    a = new int[1]
    a[0] = 100
}

```

```

main(....) {
    a[] = {10, 20, 30}
    change(a)
    Print(a[0])
}

```

```

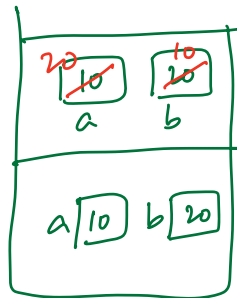
swap(a, b) {
    a ↔ b
}

```

```

a = 10
b = 20
swap(a, b)

```

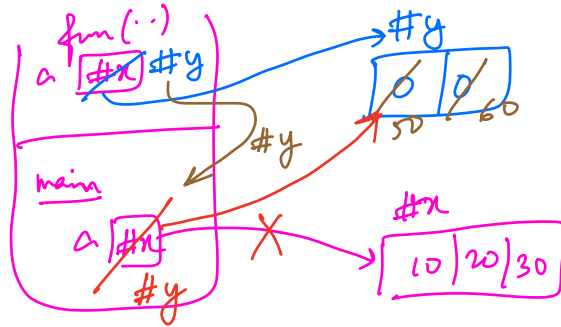


```

fun (a[]) {
  a = {0,0}
  a[0] = 50, a[1] = 60
  return a
}

main (...) {
  a → {10, 20, 30}
  a = fun(a)
  Print(a[0]) → 50
}

```



[Break till 10:37 PM]

Sorting

Arrangement of some data based on some parameter

$\{2, 3, 9, 10\} \rightarrow$ Asc order of magnitude

$\{10, 9, 3, 2\} \rightarrow$ Desc. order of magnitude

$\{1, 13, 9, 6, 12\} \rightarrow$ Asc order of no. of factors

	↓	↓	↓	↓	↓
# factors $\rightarrow$	1	2	3	4	6

$\text{sort}(\{6, 1, 4\}) \rightarrow \{1, 4, 6\}$

Java $\rightarrow$ `Arrays.sort(a);`
`Collections.sort(arraylist);`

Python $\rightarrow$ `a.sort()`

Q1) Given $arr[n]$, minimize the total cost of emptying the array. The cost of removing an element is defined as the sum of the remaining elements in the array at the time of removal.

$$arr \rightarrow \{2, 1, 4\}$$

$$\begin{array}{l} \text{Remove } 2 \xrightarrow{\{2, 1, 4\}} 2+1+4 = 7 \\ \text{Remove } 1 \xrightarrow{\{1, 4\}} 1+4 = 5 \\ \text{Remove } 4 \xrightarrow{\{4\}} 4 = 4 \\ \hline 16 \end{array}$$

$$\begin{array}{l} \text{Remove } 4 \xrightarrow{\{2, 1, 4\}} 2+1+4 = 7 \\ \text{Remove } 2 \xrightarrow{\{2, 1\}} 2+1 = 3 \\ \text{Remove } 1 \xrightarrow{\{1\}} 1 = 1 \\ \hline 11 \end{array}$$

$$arr \rightarrow \{4, 6, 1\}$$

$$\begin{array}{l} \text{Remove } 6 \rightarrow 4+6+1 = 11 \\ \text{Remove } 4 \rightarrow 4+1 = 5 \\ \text{Remove } 1 \rightarrow 1 = 1 \\ \hline 17 \end{array}$$

$$arr \rightarrow \{3, 5, 1, -3\}$$

$$\begin{array}{l} \text{Remove } 5 \rightarrow 5+3+1+(-3) \\ \text{Remove } 3 \rightarrow 3+1+(-3) \\ \text{Remove } 1 \rightarrow 1+(-3) \\ \text{Remove } -3 \rightarrow (-3) \end{array}$$

$$(5*1) + (3*2) + (1*3) + ((-3)*4)$$

sort

$$\{-3, 1, 3, 5\}$$

$$\begin{array}{l} \leftarrow 5*1 \\ + 3*2 \\ + 1*3 \\ + (-3)*4 \end{array}$$

$\text{sort}(a) \longrightarrow O(n \cdot \log n)$ T.C.

$k=1, \text{ans}=0$

for $i \rightarrow n-1$ to 0 {

$\text{ans} += a[i] * k$

$k += 1$

}

T.C $\rightarrow O(n \cdot \log n)$

S.C $\rightarrow O(n)$

Q2) Given $a[n]$ of distinct elements, find the count of noble integers.

$a[i]$ is noble if the count of elements in $a[i]$ that are smaller than $a[i]$ is equal to $a[i]$.

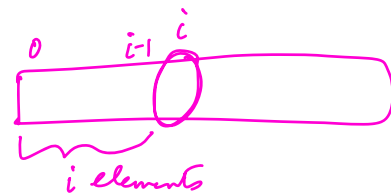
$a \rightarrow \{1, -5, 3, 5, -10, 4\}$
No. of elements smaller than $a[i] \rightarrow 2 \quad 1 \quad 3 \quad 5 \quad 0 \quad 4$

Ans $\rightarrow 3$.

$i \rightarrow 0 \quad 1 \quad 2 \quad 3$
 $a \rightarrow \{-3, 0, 2, 5\}$

Ans $\rightarrow 1$.

count of smaller $\rightarrow 0 \quad 1 \quad 2 \quad 3$



if a is sorted in ASC order,

$a[i]$ is greater than i elements in the array.

$\Rightarrow a[i]$ is noble iff $a[i] == i$.

sort(arr)

cnt = 0

for $i \rightarrow 0$ to $n-1$ {

if (arr[i] == i)

cnt += 1

}

$O(n \log n)$ T.C

$O(1)$ S.C. except the sorting part.

not necessarily distinct

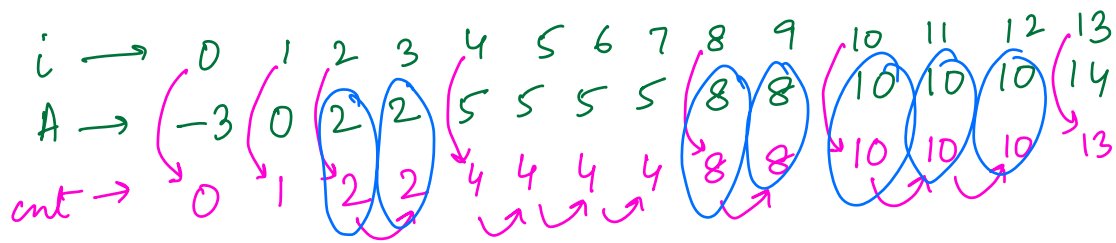
Q3) Given arr[n] of n elements, find the count of noble integers.

arr[i] is noble if the count of elements in arr[i] that are smaller than arr[i] is equal to arr[i].

$i \rightarrow$ 0 1 2 3 4
arr $\rightarrow$ { -10, 1, 1, 3, 100 } Ans $\rightarrow$ 3.
cnt $\rightarrow$ 0 1 1 3 4

$i \rightarrow$ 0 1 2 3 4 5 6 7 8
arr $\rightarrow$ { -10, 1, 1, 2, 4, 4, 4, 8, 10 } Ans $\rightarrow$ 5.
cnt $\rightarrow$ 0 1 1 3 4 4 4 7 8
0 1 1 3 4 4 4 7 8
i i i i i i i i i

< 4
4 4 4 4
i i i i i



Ans $\rightarrow 7$.

fn count-noble-integers(arr, n) {

 sort(arr)

 ans = 0

 c = 0

 for i $\rightarrow$ 0 to n-1 {

 if (i == 0 or arr[i] != arr[i-1]) {

 c = i

 }

 if (c == arr[i])

 ans += 1

 }

 return ans

}

$O(n \log n)$ TC.

$O(1)$ SC. except sorting.